



Chapitre 4 : MongoDB

Opérations d'agrégation

Ines Slimen

► Agrégation



- Les opérations d'agrégation :
 - Traitent les enregistrements de données et renvoient les résultats calculés
 - Regroupent les valeurs de plusieurs documents et peuvent effectuer diverses opérations sur les données groupées pour retourner un seul résultat
- Le framework d'agrégation de MongoDB est basé sur le concept de pipelines de traitement des données:
 - Les documents entrent dans un pipeline à plusieurs étapes ou stages qui transforment les documents et renvoient un seul résultat final





► Méthode *aggregate()*

- L'agrégation se fait via la méthode
`db.collection.aggregate( [ { <stage> }, ... ] )`
- Les stages du pipeline apparaissent dans un tableau
- Chaque stage agit sur la sortie délivrée par le stage qui le précède
- Chaque stage est représenté sous la forme d'un document et possède un opérateur
- Un opérateur peut utiliser des fonctions (de manipulation de chaîne de caractère, de comparaison,...)
- La plupart des stages peuvent être utilisés plusieurs fois
 - Ce n'est pas le cas du **\$out**, par exemple

► Exemple d'un pipeline avec *aggregate()*

Collection

```
db.orders.aggregate( [  
  $match stage → { $match: { status: "A" } },  
  $group stage → { $group: { _id: "$cust_id", total: { $sum: "$amount" } } }  
)
```

{ cust_id: "A123", amount: 500, status: "A" }
{ cust_id: "A123", amount: 250, status: "A" }
{ cust_id: "B212", amount: 200, status: "A" }
{ cust_id: "A123", amount: 300, status: "D" }

orders

\$match

{ cust_id: "A123", amount: 500, status: "A" }
{ cust_id: "A123", amount: 250, status: "A" }
{ cust_id: "B212", amount: 200, status: "A" }

\$group

{ _id: "A123", total: 750 }
{ _id: "B212", total: 200 }

Source : <https://docs.mongodb.com/manual/core/aggregation-pipeline/#pipeline-expressions>



► Exemples de stages (1/2)

- **\$project** : Transmet les documents selon les champs demandés (éliminer certains et en garder d'autres) à l'étape suivante du pipeline. Ces champs peuvent être déjà existants à l'entrée ou nouvellement calculés
- **\$match** : Filtrer les documents pour ne transmettre à la prochaine étape du pipeline que ceux qui correspondent aux conditions spécifiées
- **\$sort** : Trie tous les documents d'entrée et les renvoie au pipeline
- **\$limit** : Limite le nombre de documents passés à l'étape suivante du pipeline



► Exemples de stages (2/2)

- **\$skip** : Ignore le nombre spécifié de documents et passe les documents restants à l'étape suivante du pipeline
- **\$group** : Équivalent du group by en SQL, permet de créer des groupements et d'appliquer des fonctions de groupe (max, avg, ...)
- **\$unwind** : Déconstruit un champ de type tableau à partir des documents d'entrée pour avoir en sortie un document correspondant à un élément du tableau
- **\$lookup** : permet de récupérer un document référencé par le document courant (une sorte de jointure)
- **\$out** : Écrit les documents résultants de l'agrégation dans une collection. Doit être la dernière étape

► \$Project (1/4)



- Manipulation de chaines de caractères

Exemples de fonctions	Exemples
\$toLower/ \$toUpper	<code>db.livres.aggregate([{ \$project: { _id: 0, auteur: { \$toLower: "\$author" }, title: 1 } }])</code>
\$concat	<code>db.livres.aggregate([{ \$project: { author: 1, title_description: { \$concat: ["\$title", "-", "\$description"] } } }])</code>
\$split	<code>db.livres.aggregate([{ \$project: { _id: 0, description: { \$split: ["\$description", " "] } } }])</code>
\$substr	<code>db.livres.aggregate([{ \$project: { _id: 0, description: { \$substr: ["\$description", 0, 4] } } }])</code>
\$strLenBytes	<code>db.livres.aggregate([{ \$project: { _id: 0, title: 1, length: { \$strLenBytes: "\$title" } } }])</code>

► \$Project (2/4)



- Manipulation des valeurs numériques

Exemples de fonctions	Exemples
\$add	db.livres.aggregate([\$project :{ _id: 0, likes:{ \$add ["\$likes",3]}}]])
\$ceil	db.livres.aggregate([\$project :{ _id: 0, likes:{ \$ceil ["\$likes"]}}]])
\$trunc	db.livres.aggregate([\$project :{ _id: 0, likes:{ \$trunc ["\$likes"]}}]])



► \$Project (3/4)

- Manipulation des tableaux

Exemples de fonctions	Exemples
\$arrayElemAt	db.livres.aggregate([{ \$project: { _id: 0, first: { \$arrayElemAt: ["\$tags", 0] } } }]])
\$reverseArray	db.livres.aggregate([{ \$project: { _id: 0, first: { \$reverseArray: ["\$tags"] } } }]])
\$size	db.livres.aggregate([{ \$project: { _id: 0, size: { \$size: ["\$tags"] } } }]])

► \$Project (4/4)



- Manipulation des dates

Exemples de fonctions	Exemples
<code>\$year</code>	<code>db.livres.aggregate([{\$project:{_id: 0, year:{\$year:"\$date_pub"}}}])</code>
<code>\$month</code>	<code>db.livres.aggregate([{\$project:{_id: 0, month:{\$month:"\$date_pub"}}}])</code>
<code>\$isoDayOfWeek</code>	<code>db.livres.aggregate([{\$project:{_id: 0, day:{\$isoDayOfWeek:"\$date_pub"}}}])</code>

► \$match ... \$sort ... \$limit ... \$skip



- `db.livres.aggregate([ { $match: { "author": /eric/i } }, { $project: { _id: 0, title: 1, author: 1 } } ]).pretty()`
- `db.livres.aggregate([ { $project: { _id: 0, title: 1, an: { $year: "$date_pub" } } }, { $match: { an: 2010 } } ])`
- `db.livres.aggregate([ { $sort: { "date_pub": -1 } }, { $project: { _id: 0, title: 1 } } ])`
- `db.livres.aggregate([ { $project: { _id: 0, title: 1, an: { $year: "$date_pub" } } }, { $sort: { an: -1 } } ])`
- `db.livres.aggregate([ { $sort: { "date_pub": -1 } }, { $project: { _id: 0, title: 1, date_pub: 1 } }, { $skip: 1 }, { $limit: 2 } ])`

▶ \$group (1/4)



- **\$group** joue le même rôle que le group by au niveau d'une requête SQL
- \$group prend deux paramètres:
 - **_id** : critère de groupement, peut être vide, simple ou composé
 - Le deuxième paramètre : fonctions de groupe à appliquer à un ensemble de documents ou à tous les documents lorsque **_id** est vide

- Afficher le nombre de livre par **auteur** :

```
db.livres.aggregate([{$group:{_id:"$author",nbr_livre:{$sum:1}}}] )
```

```
SELECT count(*) nbr_livre FROM livres GROUP BY author
```

- Afficher le nombre de livre par **nom_auteur** et par **annee** :

```
db.livres.aggregate([{$group:{_id:{nom_auteur:"$author",  
annee:{$year:"$date_pub"}}, nbr_livre:{$sum:1}}}] )
```

```
SELECT count(*) nbr_livre FROM livres GROUP BY nom_auteur, annee
```

▶ \$group (2/4)



- `db.livres.aggregate([{$group:{_id:"$author",sum_likes:{$sum:"$likes"}}}])`

SELECT sum(likes) sum_likes FROM livres GROUP BY author

- `db.livres.aggregate([{$group:{_id:{},avg_like:{$avg:"$likes"}}}])`

SELECT avg(likes) avg_like FROM livres

- `db.livres.aggregate([{$group:{_id:{},max_like:{$max:"$likes"}}}])`

SELECT max(likes) max_like FROM livres

- `db.livres.aggregate([{$group:{_id:{$year:"$date_pub"},min_like:{$min:"$likes"}}}])`

SELECT min(likes) min_like FROM livres GROUP BY year

▶ \$group (3/4)



- *\$first* et *\$last* sont des fonctions de groupe qui permettent d'avoir, respectivement, la première et la dernière valeur de chaque groupe.
- Utilisées généralement avec la fonction \$sort.
- Afficher pour chaque année de publication, le premier auteur :

```
db.livres.aggregate([{$group:{_id:{$year:"$date_pub"},  
auteur:{ $first:"$author"}}}])
```



▶ \$group (4/4)

Construction d'un tableau :

- **\$addToSet** : Afficher la liste des livres par auteur. Pas de duplication des valeurs.

```
db.livres.aggregate([{$group:{_id:"$author",livres:{$addToSet:"$title"}}}])
```

- **\$push** : similaire à \$addToSet, mais avec duplication des valeurs.

▶ \$lookup : Récupérer un document référencé (Foreign Key)

```
db.conference.insert({  
  title: "Plein de mots-clefs hyper cools...",  
  description: "Refaites votre SI avec tout ça, sinon vous êtes mort!",  
  speaker_id: "techno_maniac"  
})
```

```
db.speaker.insert(  
  {  
    _id: "techno_maniac",  
    name: "Jean-Marcel Le Maniac",  
    email: "le_maniac@web2day.org"  
  })
```

- `db.conference.aggregate([{"$lookup": {"from": "speaker", "localField": "speaker_id", "foreignField": "_id", "as": "speaker_infos"}}])`



► \$unwind ... \$out

○ **\$unwind** : Séparer les éléments d'un tableau

- `db.livres.aggregate([{$unwind: "$tags"},
{$group: {_id: "$_id", nbre_tags: {$sum: 1}}}]);`

○ **\$out** : Récupérer le résultat d'agrégation dans une autre collection

- `db.livres.aggregate([{$unwind: "$tags"},
{$group: {_id: "$_id", nbre_tags: {$sum: 1}}},
{$out: "results"}])`

- Visualisation du résultat du stage **\$out** dans la collection **results** :
`db.results.find()`



► Options d'agrégation

- Analyse de l'agrégation : avec l'option **explain**
 - `db.livres.aggregate([{$group:{_id:"$author",nbr_livre:{$sum:1}}}],{explain:true})`
- Gestion de l'agrégation sur le disque et non dans l'espace mémoire (dans le cas où le résultat dépasse 100M)
 - `db.livres.aggregate([{$group:{_id:"$author",nbr_livre:{$sum:1}}}],{allowDiskUse:true})`